

Cahier des Charges Stratégique : Plateforme SaaS Pharmacie

Ce document consolide et rationalise les exigences pour le lancement d'une plateforme SaaS B2B dédiée aux pharmacies africaines, portée par notre vision d'éditeur de logiciels.

1. Vision et Cible

Créer la plateforme de référence pour la gestion des pharmacies en Afrique en répondant aux enjeux critiques du terrain : instabilité de la connexion internet, gestion complexe des stocks (ruptures), prolifération de faux médicaments, et un besoin urgent de traçabilité.

2. MVP (Minimum Viable Product) : Périmètre Recadré

Pour un "Time to Market" rapide, le MVP doit se concentrer sur la survie opérationnelle et financière de l'officine. Les fonctionnalités périphériques (auto-rapprovisionnement, gestion RH avancée) sont décalées à la V2 pour garantir la stabilité absolue du socle "Offline-First".

Module	Fonctionnalités Clés (MVP)
Caisse & Vente (POS)	Encaissement rapide, fonctionnement "Offline-First" avec synchronisation asynchrone dès le retour réseau.
Gestion des Stocks	Entrées/sorties, alertes de ruptures, traçabilité fine par lots.
Cycle de Vie & Pérémutations	Alertes de péremption imminente, dons, transferts inter-pharmacies, retours fournisseurs et promotions.
Administration & Audit	Tableau de bord gérant (CA, marges), gestion des fournisseurs, habilitations basiques, et journal d'audit des actions.
Localisation	Interface bilingue (Français, Anglais) prête au déploiement régional.

3. Roadmap Évolutive (V2 à V4)

- **V2 (Croissance & RH)** : Application mobile dédiée au dirigeant, intégration des paiements mobiles, pointage du personnel, portail fournisseurs, et IA pour la prévision des ventes.
- **V3 (Écosystème B2B)** : Marketplace pharmaceutique, centrale d'achat, téléconsultation, livraison, e-prescription, et gestion des visiteurs médicaux.
- **V4 (Impact Sectoriel)** : Plateforme de e-learning pour le personnel officinal, certifications, et bourse continentale.

4. Architecture Technique (SaaS Multi-Tenant)

Une stack moderne, modulaire et taillée pour l'Offline-First :

- **Frontend** : Flutter (Web pour le back-office, Mobile/Tablette pour la caisse et le gérant).
- **Backend** : NestJS (Architecture modulaire API-First).
- **Base de Données** : PostgreSQL (données chaudes multi-tenant) + SQLite (local sur l'appareil pour le sync hors-ligne).
- **Cache & Fichiers** : Redis (sessions, files d'attente), MinIO (stockage compatible S3).
- **Sécurité** : OAuth2, MFA, et journalisation immuable.

5. Prompt Directeur pour Équipe de Développement / IA

Ce prompt est conçu pour initialiser le projet via une IA de code ou briefer une équipe technique.

CONTEXTE :

Tu agis en tant que Tech Lead Sénior pour T&M Logiciels. Nous développons un SaaS B2B de gestion de pharmacies pour le marché africain (Multi-tenant, Offline-first, haute sécurité).

MISSION :

Initialise l'architecture backend (NestJS) et frontend (Flutter) pour le MVP. Le code doit être extrêmement modulaire, suivre les principes de la Clean Architecture, et être strictement typé.

SPÉCIFICATIONS TECHNIQUES :

- Backend : NestJS, TypeORM (PostgreSQL), Redis pour les files d'attente, authentification JWT avec OAuth2.
- Frontend : Flutter (Riverpod ou BLoC pour le state management), interface responsive.
- Base de données : Schéma multi-tenant robuste (séparation par `tenant\_id` avec Row Level Security).
- Synchronisation : Architecture locale via SQLite sur le device client avec synchronisation bidirectionnelle intelligente vers l'API.

TACHES IMMÉDIATES :

1. Génère la structure NestJS avec les modules essentiels : 'Auth', 'Tenant', 'Inventory', 'POS'.
2. Rédige le schéma TypeORM pour les entités : User, Role, Product, Lot, Sale. Assure-toi d'inclure les champs 'tenant_id', 'created_at', 'updated_at', et 'deleted_at' (Soft Delete).
3. Fournis l'implémentation d'un intercepteur/middleware pour la gestion des erreurs et l'injection automatique du `tenant\_id` dans toutes les requêtes.

4. Côté Flutter, génère la structure de base et un service d'abstraction pour la logique de synchronisation locale (SQLite) <-> API REST.

CONTRAINTES STRICTES :

- Approche API-First.
- Tests unitaires (Jest) obligatoires pour la logique métier.
- Documentation Swagger générée automatiquement.
- Zéro compromis sur la sécurité (Log d'audit systématique sur chaque action de mutation des données).